

UNIT 2, BUILDING INTELLIGENT AGENT WORKFLOWS

LECTURE 2

Tool Calling, in Depth

How the model reads a schema and chooses, what the execution loop really does, and the part nobody prepares for: what happens when a tool fails halfway through.

SECTION 00

A comfortable lie from Unit 1

Every tool we wrote worked perfectly. Real tools do not.

RECALL

Remember every tool from Unit 1?

```
def get_room_availability(hotel, date):  
    return f"{hotel}: {n} rooms available."    # always works
```

Every tool you wrote in Unit 1 returned a clean, useful string on the very first try. It never timed out. It never threw. It never returned garbage. That was a teaching convenience, and today we retire it.

THE UNCOMFORTABLE REALITY

A real tool reaches a real service over a real network. It will time out, it will throw, it will hit a rate limit, and worst of all it will sometimes succeed and hand back junk. Your loop has to survive every one of those without falling over, and making it do so is most of the actual work of building an agent.

Today

- How the model actually chooses a tool, and why the description matters more than the code.
- The execution loop, one more time, but this time with the failures drawn in.
- The four ways a tool fails, and why one of them is far nastier than the rest.
- A single defended executor that turns every failure into a sentence the model can read.
- The difference between a failure you retry and a failure you must not.

HOLD ON TO THIS

The running example this lecture is a weather assistant, because weather tools are the perfect thing to break. The data is trivial, so nothing distracts from the one thing we are studying: what happens when the tool itself misbehaves.

SECTION 01

How the model chooses

It never sees your code. It sees a description, and it picks from that.

The model chooses from words, not code

1

You send

The conversation, plus a list of tool descriptions. The model sees the schema. It never sees your function body.

2

It matches

The model compares the user need against each description and picks the one whose words fit best. This is a language match, not a logical proof.

3

It requests

It returns a request: this tool, these arguments. Still just text. Nothing has run.

THE CONSEQUENCE FOR YOU

If two tools have similar descriptions, the model will confuse them, and no amount of fixing the code will help, because the model never reads the code. When a tool is chosen wrongly, the bug is almost always in your English, not your Python. This was true in Unit 1. It is louder now that you have more tools.

Two descriptions the model will confuse

BAD

Overlapping descriptions

"get_weather: gets weather information."

"get_forecast: gets weather information."

The model has no way to tell these apart, and will pick more or less at random.

GOOD

Descriptions that carve cleanly

"get_weather: current conditions right now."

"get_forecast: predicted weather for a future date."

Now the words themselves tell the model when each applies.

HOLD ON TO THIS

A useful test: cover the tool names and read only the descriptions. If you cannot tell which tool to use for a given request, neither can the model. Fix the words.

SECTION 02

When tools fail

Four ways, and the one that will actually hurt you.

The four ways a tool fails

IT THROWS

The call raises an exception. A dropped connection, a bad response, a library error. Loud, and easy to catch once you know to.

IT HANGS

The call never returns. A slow service, a lost packet. It does not error, it just sits there, and your whole request sits with it.

IT RATE LIMITS

It works, then starts refusing. Call it too fast and it returns 429. Retrying blindly makes it angrier, not happier.

THE LAST ONE IS THE ONE THAT GETS YOU

A tool that throws, you catch. A tool that hangs, you time out. But a tool that succeeds and hands back an error page as if it were data, that flows straight into the model, which cheerfully reasons over garbage and gives your user a confident, wrong answer. No error appears anywhere.

One tool, four ways to break it

```
def get_weather_that_throws(city):  
    raise ToolError("Connection reset by peer.")  
  
def get_weather_that_times_out(city):  
    time.sleep(2.0); return ...    # hangs first  
  
def get_weather_that_returns_junk(city):  
    return "\x00 <html>502 Bad Gateway</html>"    # lies  
  
def get_weather_that_rate_limits(city):  
    # works twice, then raises 429
```

These four live in the notebook. Same weather data underneath, four different ways of failing to deliver it. We are about to write one function that survives all four.

SECTION 03

The defended executor

One function, four defences, no crashes.

The one function that has to be bulletproof

```
def call_tool(name, args, registry, retries=2):  
  
    if name not in registry:           # 1 whitelist  
        return ToolResult(ok=False, ...)  
  
    for attempt in range(retries + 1):  # 2 retry  
        try:  
            result = func(**args)  
            if not looks_usable(result): # catch the lie  
                return ToolResult(ok=False, ...)  
            return ToolResult(ok=True, ...)  
        except Exception as e:         # 3 catch
```

THE PROMISE THIS FUNCTION MAKES

Whatever happens, it returns a ToolResult with a readable observation. Never an exception, never a null byte, never a hang that takes the request down. The model always gets a sentence it can reason about, even when that sentence is "this failed, proceed without it".

Defence four is the subtle one

```
def looks_usable(text):  
    if not isinstance(text, str): return False  
    if not text.strip():          return False    # empty  
    if "\x00" in text:            return False    # null bytes  
    if "502" in text and "gateway" in text.lower():  
        return False              # error page  
    return True
```

WHY THIS CHECK EARNS ITS KEEP

The other three failures announce themselves. This one does not. Without this check, an error page reaches the model as if it were weather data, and the model tells your user it is 502 degrees in Bad Gateway. A cheap sanity check on tool output is the only thing standing between a silent tool failure and a confident wrong answer.

The failure you must not retry

TRANSIENT, RETRY IT

A dropped connection

It might work next time. A network blip, a momentary overload. Wait a moment, with a growing pause, and try again. This is what retry is for.

PERMANENT, DO NOT RETRY

Wrong arguments

Calling `get_weather` with no city will fail identically every time. Retrying it three times just wastes three calls and delays the honest failure. Report it at once.

THE DISTINCTION IN CODE

Our executor catches a `TypeError`, which means bad arguments, and returns immediately without retrying. Everything else it treats as possibly transient and retries with a growing pause. Telling these two apart is the difference between a system that recovers gracefully and one that hammers a dead endpoint and calls it resilience.

BACKOFF

Why a growing pause, not a fixed one

FIXED RETRY

Retry every 0.1 seconds

If a service is overloaded, forty clients all hammering it every tenth of a second keep it overloaded. You are part of the problem you are trying to survive.

BACKOFF

Wait longer each time

Half a second, then one, then two. You give the service room to recover, and forty clients backing off together let it breathe. This is standard practice for a reason.

HOLD ON TO THIS

The rate limiting tool from earlier is exactly why. Hit a 429 and retry instantly and you get another 429. Back off, and the limit clears. Politeness, encoded as arithmetic.

SECTION 04

The loop, hardened

Where the defended tool call meets the agent loop.

The Unit 1 loop, with the failures drawn in

```
for step in range(max_steps):
    message = client.chat.completions.create(...)
    if not message.tool_calls:
        return message.content          # done

    for call in message.tool_calls:
        outcome = call_tool(call.name, args)  # defended
        messages.append({"role": "tool",
                          "content": outcome.observation})
```

NOTICE HOW LITTLE THE LOOP CHANGED

It is almost exactly your Unit 1 loop. The one difference is that the raw function call became `call_tool`, and because `call_tool` never raises and always returns a readable observation, the loop needs no new error handling of its own. Push the mess down into one well defended function, and everything above it stays simple. That is good engineering, not just here but everywhere.

Watch the model recover, live

```
>>> run_with_tools(client, MODEL, "Weather in Delhi?",  
...               registry={"get_weather": get_weather_that_throws})
```

```
[step 1] get_weather({city: Delhi}) [FAILED]  
    -> Tool failed after 3 attempts. Proceed without it.
```

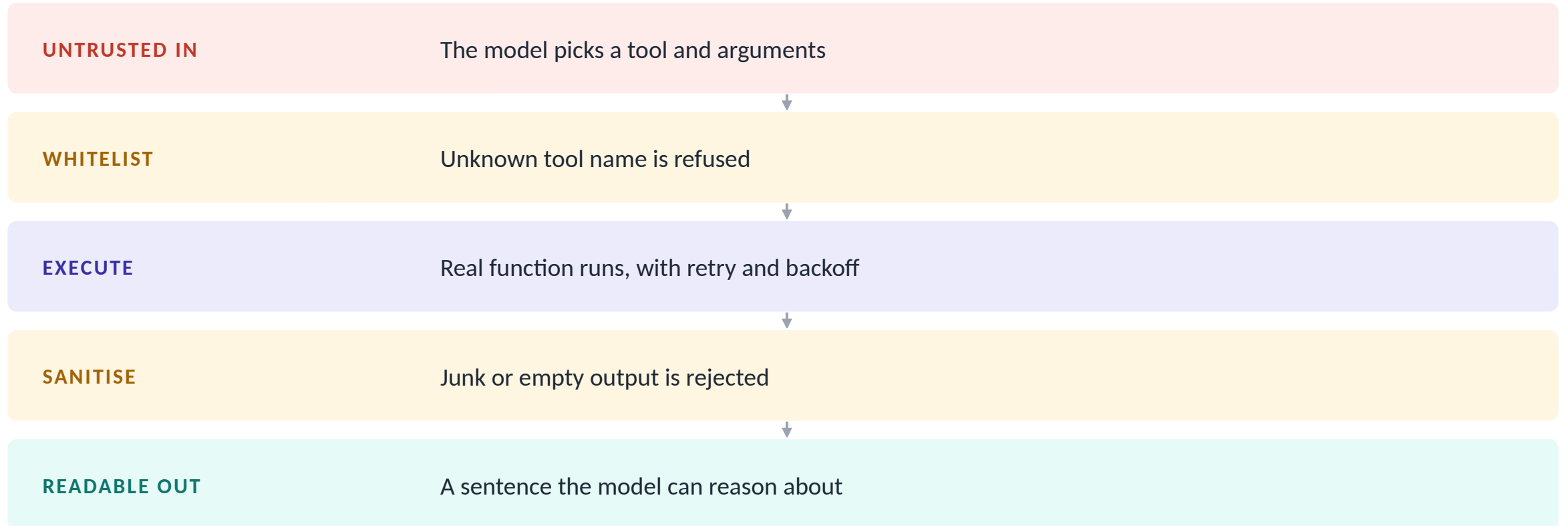
```
[step 2] done
```

```
"I could not reach the weather service for Delhi just now,  
so I do not have current conditions. Please try again."
```

THE BEHAVIOUR WE WANTED

The tool failed three times. The loop did not crash. The failure went back to the model as a readable sentence, and the model did the honest thing: it told the user it could not find out, rather than inventing a temperature. That honesty was made possible entirely by how we handled the failure, not by anything clever the model did.

The whole chain, one picture



Untrusted input goes in the top. A clean, readable observation comes out the bottom. Everything dangerous is contained in the middle.

Hold on to this

The loop does not trust the model, and it does not trust the tools either.

The model chooses from words

Confusable descriptions cause bugs no code fix can reach.

Four failure modes, one is silent

A tool that lies is worse than one that crashes.

One function absorbs the mess

`call_tool` never raises and always returns a readable string.

Retry transient, never permanent

And back off, so you help the service instead of hammering it.

Before next session

DO

Break your own tool

Take a tool from your Practical 2 agent. Write three broken versions: one that throws, one that returns an empty string, one that returns junk. Run each through `call_tool` and confirm none of them crashes your loop.

DO

Add a fifth failure

The four in the notebook are not the only ones. Invent a fifth realistic failure mode and add a defence for it to `call_tool`. Write one sentence on whether it is transient or permanent.

THINK

The honest answer

When a tool fails and the model cannot answer, what should your specific agent say to the user? Write the exact sentence. Vague failure messages are their own kind of bug.

ON THE FIRST ONE

This is the exercise that will save you in your capstone. A tool that crashes in a demo is embarrassing. A tool that crashes in front of an assessor, when a single try-except would have caught it, is avoidable. Build the habit now.

Next session

Unit 2, Lecture 3

API Integration

Today the tools were functions on your own machine. Next time they reach out to real services over the network, which is where every failure mode from this lecture stops being a teaching example and becomes Tuesday afternoon. Real timeouts, real rate limits, real credentials, and how to wrap a messy external API into a tool an agent can use safely.

COME WITH

Your three broken tools tested, your fifth failure mode added, and the exact sentence your agent says when it fails.